JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY
SECTOR 128, NOIDA
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
ACADEMIC YEAR: 2026–2027

PROJECT SYNOPSIS

AEROPATH: EMERGENCY DRONE FLEET MANAGEMENT SYSTEM



Java Project-Based Learning (PBL)

SUBMITTED TO:
Dr. Prakhar Mishra
Faculty Mentor
Department of Computer Science & Engineering
JIIT, Sector 128, Noida

SUBMITTED BY:
Madhav Kansal (992501030034)
Deepanshi Goswami (992501030037)
Akshita Gupta (992501030047)
Priyan Saxena (992501030477)

1. TITLE OF THE PROJECT

AeroPath: Emergency Drone Fleet Management System

2. INTRODUCTION

In emergency medical healthcare, time-to-delivery for critical medical supplies such as whole blood, antivenoms, emergency vaccines, cardiac defibrillators, and organs for transplant is often severely compromised by urban road congestion, geographic barriers, remote rural terrain, or natural disasters. Autonomous medical drone fleets offer a revolutionary aerial logistics model, establishing direct point-to-point airborne supply corridors between central hospital hubs and remote health clinics or emergency rescue sites.

AeroPath is a Java-based platform that automates these drone fleets. Built with computer science concepts, it handles priority scheduling, battery and payload validation, dynamic routing (Dijkstra/A*), multithreaded tracking, custom error handling, and database storage—using a spatial grid to simulate flights without physical GPS hardware.

3. PROBLEM STATEMENT

Current emergency medical drone systems have five main flaws:

1. **Poor Prioritization:** Critical items (like blood or antivenom) are processed on a first-come basis instead of treated as top priority over routine deliveries.
2. **Manual Errors:** Human operators risk sending drones with payloads that are too heavy or batteries too low for a safe return trip.
3. **Basic Routing:** Flights use simple straight lines rather than adjusting for distance, air traffic, or bad weather.
4. **No Multi-Drone Tracking:** Systems can't track multiple active flights at once, leading to delayed updates during emergencies.
5. **Hidden Maintenance Needs:** Lack of automatic flight logging makes it easy to miss worn-out parts, leading to unexpected failures.

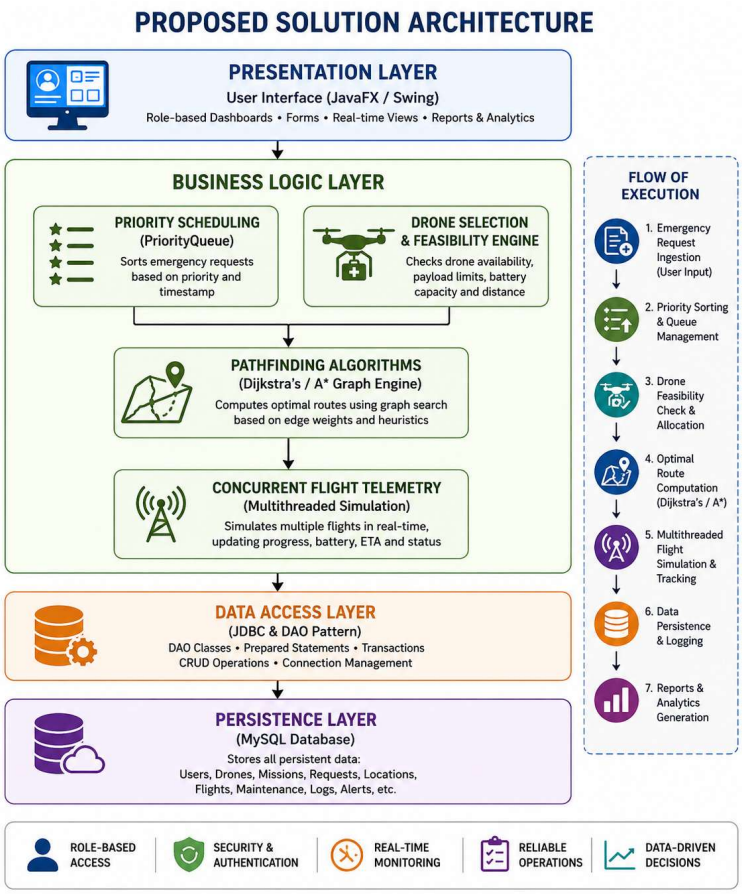
4. OBJECTIVES

- **Automated Lifecycle Management:** Java desktop platform automating emergency drone missions from request ingestion to proof-of-delivery.
- **Intelligent Priority Scheduling:** Priority queue engine dynamically ranking requests using urgency weights and submission timestamps.
- **Algorithmic Feasibility Engine:** Mathematical validator checking payload limits, discharge rates, distance consumption, and battery reserves pre-launch.

- **Optimal Path Generation:** Dijkstra and A* graph search algorithms operating on an adjacency-list aerial network.
- **Multithreaded Concurrent Tracking:** Multithreaded simulation (Runnable) streaming real-time telemetry, ETAs, and battery usage across active flights.
- **Persistence & Security:** Role-based access control and persistent storage built on the Data Access pattern.

5. PROPOSED SOLUTION / METHODOLOGY

AeroPath adopts a clean 3-Layer Software Architecture consisting of the Presentation Layer (JavaFX / Swing), Business Logic Layer (Core Computational Algorithms & Multithreaded Engine), and Data Access & Persistence Layer (JDBC & MySQL Database).



5.1 Complete System Execution Flow

The system operates through an automated 12-step execution (Tentative) :

Emergency Request Ingestion → Validation → Priority Calculation → Drone Search → Battery & Payload Feasibility Check → Drone Selection → Route Planning → Mission Creation →

Multithreaded Telemetry Tracking → Delivery Confirmation → Flight Logging → Maintenance Evaluation

5.2 Mathematical Specifications & Algorithmic Design

A. Priority Scheduling Algorithm

Emergency medical requests are ingested and held in a PriorityQueue driven by a specialized comparator based on a calculated priority score, P_score. The priority score of a request is evaluated as:

$$P_{score} = W_{priority} \times 10^5 - (t_{current} - t_{submission})$$

$$\text{where } W_{priority} = \begin{cases} 4, & \text{CRITICAL (Antivenom, Cardiac, Blood)} \\ 3, & \text{HIGH (Emergency Vaccines, Defibrillators)} \\ 2, & \text{MEDIUM (Surgical Supplies, Diagnostics)} \\ 1, & \text{LOW (Routine Pharmaceuticals)} \end{cases}$$

This ensures urgent medical tiers take absolute precedence, while a time-decay factor prevents older requests within the same tier from waiting indefinitely.

B. Drone Selection & Feasibility Engine

To fulfill an emergency request, candidate drones must be in the AVAILABLE state and meet two mandatory conditions:

$$\text{1. Payload Constraint:} \quad W_{payload} \leq C_{drone_max}$$

$$\text{2. Energy Constraint:} \quad E_{required} \leq B_{current}$$

The battery percentage required for a round-trip distance ($d_{\text{round_trip}} = 2 \times d_{\text{one_way}}$) accounts for the payload on the outbound leg and an empty return leg:

$$E_{required} = E_{base} + d_{round_trip} \cdot (\varepsilon_{flight} + \gamma \cdot W_{payload}) + E_{reserve}$$

where ε_{flight} is base discharge rate per km, γ is payload-drag coefficient, and $E_{reserve} = 15\%$ safety margin.

When multiple candidate drones pass both feasibility checks, the optimal drone is selected using a weighted ranking score:

$$S_{drone} = \alpha \cdot \left(\frac{B_{current}}{100}\right) + \beta \cdot \left(1 - \frac{W_{payload}}{C_{drone_max}}\right) - \delta \cdot d_{base_to_request}$$

C. Graph-Based Route Planning Engine

The spatial navigation area is modelled as a weighted directed graph $G = (V, E, w)$, where nodes (V) are locations, edges (E) are flight paths, and weights (w) represent travel distance or time.

• Dijkstra's Shortest Path Algorithm: Finds the absolute minimum-weight path across the graph using a Min-Heap priority queue to iteratively relax edge weights:

$$\text{Relaxation Step: if } d(u) + w(u,v) < d(v) \implies d(v) = d(u) + w(u,v), \quad \pi(v) = u$$

$$\text{Time Complexity: } O((|V| + |E|) \log |V|)$$

A* Search Algorithm: Speeds up pathfinding toward a specific destination by adding an admissible Euclidean heuristic $h(n)$ to the exact cost $g(n)$:

$$f(n) = g(n) + h(n)$$

$$h(n) = \sqrt{(x_{target} - x_n)^2 + (y_{target} - y_n)^2}$$

D. Multithreaded Telemetry Simulation Engine

Each active flight mission spawns an independent background execution thread implementing Java's Runnable interface. During each simulation tick interval Δt, the thread updates telemetry dynamics asynchronously:

$$\text{Distance Traveled: } d_{traveled}(t + \Delta t) = d_{traveled}(t) + v_{drone} \cdot \Delta t$$

$$\text{Progress Percentage: Progress}(\%) = \left(\frac{d_{traveled}}{d_{total}} \right) \times 100$$

$$\text{Dynamic ETA: } t_{ETA} = \frac{d_{total} - d_{traveled}}{v_{drone}}$$

$$\text{Battery Depletion Tick: } B_{current}(t + \Delta t) = B_{current}(t) - \left(\frac{E_{required}}{d_{total}/v_{drone}} \right) \cdot \Delta t$$

5.3 Object-Oriented Design & System Architecture

A. Core OOP Principles Applied:

- Encapsulation: Domain models (Drone, Mission, EmergencyRequest, Location, User) protect operational state behind private fields with explicit getters, setters, and state-validation mutators.

- Inheritance & Polymorphism: Base User class is extended into role-specific sub-classes (Admin, Dispatcher, Medical Officer, Operator, MaintenanceStaff) that override behavioural methods.
- Abstraction & Interfaces: Clean operational contracts defined via interfaces like Trackable (telemetry streaming), Maintainable (service checks), and Deliverable (proof-of-delivery).

B. Domain-Specific Custom Exception Hierarchy:

To preserve operational safety, custom runtime exceptions are handled gracefully:

- InsufficientBatteryException: Thrown when candidate battery is below the required round-trip safety threshold ($E_{\text{required}} > B_{\text{current}}$).
- PayloadExceededException: Thrown when payload weight exceeds a drone's max capacity rating.
- DroneUnavailableException: Thrown when no operational drones are in the AVAILABLE state.
- RouteUnavailableException: Thrown when graph pathfinding fails to find a connected path.
- InvalidMissionException: Thrown on illegal state transitions during mission execution.

C. Role-Based Access Control (RBAC) Matrix

System Role	Request Ingestion	Dispatch / Launch	Live Telemetry	Maintenance	User / System Admin
Medical Officer	Yes	No	Read-Only	No	No
Dispatcher	Yes	Yes	Full	No	No
Drone Operator	No	Yes	Full	No	No
Maintenance Staff	No	No	Read-Only	Full	No
Admin	Yes	Yes	Full	Full	Full

6. EXPECTED OUTCOMES

By the end of the project, we hope the system will deliver:

- **Interactive Desktop GUI:** JavaFX/Swing frontend for mission control, access control, and telemetry tracking.
- **Automated Request Prioritization:** Priority queue sorting based on medical urgency and submission time.
- **Feasible Drone Matching Engine:** Automated pre-flight checks for payload, discharge rates, and battery margins.
- **Optimized Flight Path Routing:** Dijkstra and A* graph routing to minimize transit time and power draw.
- **Concurrent Real-Time Mission Simulation:** Multithreaded engine tracking live positions, ETAs, and battery usage.
- **Delivery, Maintenance & Analytics System:** DAO database persistence for flight logs, delivery records, and maintenance alerts.

7. TIMELINE / WORK PLAN

Phase	Milestone / Task Description	Target Completion
Phase 1	Problem definition, requirement analysis, and initial project synopsis formulation.	28–30 September 2026
Phase 2	Final synopsis review, faculty consultation, and formal project submission.	1–3 October 2026
Phase 3	Database schema design, JDBC connection pooling, DAO layer implementation, and core OOP domain models.	4–12 October 2026
Phase 4	Priority Scheduling queue engine, Comparator setup, and Feasibility Validation rules.	13–19 October 2026
Phase 5	Graph engine implementation: Adjacency list, Dijkstra's algorithm, A* algorithm, and pathfinding module.	20–27 October 2026
Phase 6	Multithreaded simulation engine (Runnable telemetry threads) and GUI interface integration.	28 Oct–5 November 2026
Phase 7	End-to-end integration testing, custom exception handling validation, performance benchmarking, and final presentation.	15–25 November 2026

8. REFERENCES

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. (For Dijkstra and A* Search Algorithm implementations).
2. Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional. (For Java design patterns, collections, and concurrency principles).
3. Oracle Corporation. The Java Tutorials: Concurrency and Multithreading API. Oracle Java Documentation.
4. Oracle Corporation. The Java Tutorials: JDBC Database Access & DAO Pattern. Oracle Java Documentation.
5. Schildt, H. (2018). Java: The Complete Reference (11th ed.). McGraw-Hill Education.